

■ Python Revision Sheet

All Basic Topics · Definitions · Examples · Code

By Kuldeep Parmar | VIT Bhopal

01 PRINT & VARIABLES

■ DEFINITION

- `print()` displays output on screen.
- Variables store data. Python automatically detects the type.
- Common types: `str` (text), `int` (number), `float` (decimal)

■ EXAMPLE

```
name = "Rahul" # string variable
age = 18 # integer variable
print(name) # Output: Rahul
print(age) # Output: 18
```

■■ CODE

```
print("Hi! Kuldeep Parmar")

age = 20

study = "vit bhopal"

print("Kuldeep's age is:", age, "and He study in", study)
```

02 INPUT FROM USER

■ DEFINITION

- `input()` reads text typed by the user from the keyboard.
- It always returns a `STRING`, no matter what is typed.
- Store the result in a variable to use it later.

■ EXAMPLE

```
city = input("Enter your city: ")

print(city) # prints whatever the user typed
```

■■ CODE

```
enter = input("what is your purpose? ")

print(enter)
```

03 SUM OF TWO NUMBERS (Type Casting)

■ DEFINITION

- `input()` always gives a string — you must convert it for math.
- `int()` converts string to integer e.g. `int('5') → 5`
- `float()` converts string to decimal e.g. `float('3.14') → 3.14`
- This conversion is called TYPE CASTING.

■ EXAMPLE

```
x = input("Enter: ") # x = "5" (string)
x = int(x) # x = 5 (integer)
print(x + 3) # Output: 8
```

■■ CODE

```
firstnum = input("enter first number:")
secondnum = input("enter second number:")
total = int(firstnum) + int(secondnum)
print("the sum of two number is:", total)
```

04 STRING METHODS

■ DEFINITION

- `.upper()` converts all letters to UPPERCASE
- `.lower()` converts all letters to lowercase
- `.title()` capitalizes First Letter Of Each Word
- `.replace(a, b)` replaces part a of the string with b

■ EXAMPLE

```
s = "hello world"
s.upper() # "HELLO WORLD"
s.title() # "Hello World"
s.replace("hello", "hi") # "hi world"
```

■■ CODE

```
name = "kuldeep parmar"
print(name.upper())
print(name.lower())
print(name.title())
print(name.replace("kuldeep", "kuldip"))
```

05 STRING FINDING

■ DEFINITION

- `.find(sub)` returns the INDEX where substring is first found.
- Index starts from 0. Returns -1 if not found.
- Useful to check if a word exists and where it is.

■ EXAMPLE

```
s = "hello"

s.find("e") # 1 (e is at position 1)

s.find("z") # -1 (z not found)
```

■■ CODE

```
name = "kuldeep parmar"

print(name.find("parmar")) # index of "parmar"

print(name.find("u")) # index of first "u"
```

06 MEMBERSHIP OPERATOR (in)

■ DEFINITION

- 'in' checks if a value EXISTS inside a string or list.
- Returns True if found, False if not found.
- Very useful for searching without using `find()`.

■ EXAMPLE

```
"cat" in "concatenate" # True

"z" in "hello" # False
```

■■ CODE

```
name = "kuldeep parmar"

print("a" in name) # True

print("z" in name) # False
```

07 ARITHMETIC & COMPARISON OPERATORS

■ DEFINITION

- + Addition - Subtraction * Multiplication / Division
- % Modulus (remainder) ** Exponent (power) // Floor division
- Comparison: > < == != >= <= — these return True or False

■ EXAMPLE

```
10 % 3 # 1 (remainder of 10/3)
2 ** 3 # 8 (2 to the power 3)
7 // 2 # 3 (floor of 3.5 is 3)
5 > 3 # True
```

■■ CODE

```
a, b = 10, 5
print(a + b, a - b, a * b, a / b)
print(a % b, a ** b, a // b)
print(a > b, a == b, a != b)
```

08 OPERATOR PRECEDENCE (BODMAS)

■ DEFINITION

- Python evaluates expressions in this order (high to low):
- 1. () Parentheses 2. ** Exponent
- 3. * // % Multiply/Divide 4. + - Add/Subtract

■ EXAMPLE

```
10 + 5 * 2 # 20 (5*2 first, then +10)
(10+5) * 2 # 30 (brackets first)
10+5*2-3 # 17 (5*2=10, 10+10=20, 20-3=17)
```

■■ CODE

```
result1 = 10 + 5 * 2 # 20
result2 = (10 + 5) * 2 # 30
result3 = 10 + 5 * 2 - 3 # 17
print(result1, result2, result3)
```

09 LOGICAL OPERATORS

■ DEFINITION

- and → True only if BOTH conditions are True
- or → True if AT LEAST ONE condition is True
- not → REVERSES the result (True→False, False→True)

■ EXAMPLE

```
x = 8  
  
x > 5 and x < 10 # True (both true)  
  
x > 5 or x > 100 # True (first is true)  
  
not(x > 5) # False (reverses True)
```

■■ CODE

```
x, y = 10, 5  
  
print(x > 5 and y < 10) # True  
  
print(x > 5 or y > 10) # True  
  
print(not(x > 5)) # False
```

10 IF-ELSE STATEMENT

■ DEFINITION

- Used for DECISION MAKING in programs.
- If the condition is True → runs the if block.
- If the condition is False → runs the else block.

■ EXAMPLE

```
marks = 45  
  
if marks >= 50:  
  
    print("Pass")  
  
else:  
  
    print("Fail") # Output: Fail
```

■■ CODE

```
age = 20  
  
if age >= 18:  
  
    print("you are an adult")  
  
else:  
  
    print("you are a minor")
```

11 IF-ELIF-ELSE STATEMENT

■ DEFINITION

- elif (else if) is used when there are MORE THAN TWO conditions.
- Python checks top to bottom and runs the FIRST true block.
- 'else' runs only if NO condition is True.

■ EXAMPLE

```
score = 75

if score >= 90: print("A")

elif score >= 60: print("B") # Output: B

else: print("C")
```

■■ CODE

```
age = int(input("enter your age:"))

if age >= 18 and age < 60:

    print("you are an adult")

elif age <= 13:

    print("you are a child")

else:

    print("you are a senior citizen")
```

12 SIMPLE CALCULATOR (Applying if-elif-else)

■ DEFINITION

- Practical use of if-elif-else combining input + type casting + conditions.
- Checks which operator the user entered and performs that operation.
- Uses == to compare strings (the operator entered by user).

■ EXAMPLE

```
Enter: 10 + 5 → Output: 15

Enter: 8 / 2 → Output: 4.0

Enter: 10 @ 5 → Output: invalid operator
```

■■ CODE

```
first = input("first number:")

operator = input("operator (+,-,*,/):")

second = input("second number:")

if operator == "+": print(int(first) + int(second))

elif operator == "-": print(int(first) - int(second))

elif operator == "*": print(int(first) * int(second))

elif operator == "/": print(int(first) / int(second))

else: print("invalid operator")
```

13 WHILE LOOP

■ DEFINITION

- Repeats a block of code AS LONG AS a condition is True.
- Checks condition BEFORE each run.
- Always update the loop variable — or it runs forever (infinite loop)!

■ EXAMPLE

```
i = 1

while i <= 3:

    print(i) # Output: 1 2 3

    i += 1
```

■■ CODE

```
i = 1

while i <= 5:

    print(i)

    i += 1
```

14 PATTERN PRINTING (while loop)

■ DEFINITION

- Multiplying a string by a number repeats it that many times.
- `3 * "*"` gives `***`
- Combine this with a loop to print growing patterns.

■ EXAMPLE

```
print(1 * "#") # *
print(3 * "#") # ***
print(5 * "#") # *****
```

■■ CODE

```
i = 1

while i <= 5:

    print(i * "#") # *, **, ***, ****, *****

    i += 1
```

15 FOR LOOP & range()

■ DEFINITION

- for loop iterates over a sequence a known number of times.
- `range(n)` → 0 to n-1
- `range(start,end)` → start to end-1

■ EXAMPLE

```
for i in range(3):  
    print(i) # 0 1 2  
  
for i in range(1, 4):  
    print(i) # 1 2 3
```

■■ CODE

```
for i in range(6):  
    print(i) # 0 1 2 3 4 5  
  
for i in range(1, 6):  
    print(i * " ") # * ** *** **** *****
```

16 LIST

■ DEFINITION

- Ordered, changeable collection using [] brackets.
- Index starts from 0. Negative index -1 means last item.
- list[start:end] → slice (start to end-1)
- .append(x) .insert(i,x) .remove(x) len(list)

■ EXAMPLE

```
colors = ["red", "green", "blue"]  
colors[0] # "red"  
colors[-1] # "blue" (last)  
colors[0:2] # ["red", "green"]  
colors.append("yellow") # adds to end
```

■■ CODE

```
fruits = ["apple", "banana", "orange"]  
print(fruits[0], fruits[-1]) # apple orange  
print(fruits[0:2]) # [apple, banana]  
fruits.append("grape")  
fruits.insert(1, "kiwi")  
fruits.remove("banana")  
print(len(fruits))
```

17 BREAK & CONTINUE

■ DEFINITION

- Loop control statements that change how a loop behaves:
- break → EXITS the loop completely when condition is met.
- continue → SKIPS current iteration, loop continues with next item.

■ EXAMPLE

```
for i in range(5):  
    if i == 3: break # prints: 0 1 2  
  
    print(i)  
  
for i in range(5):  
    if i == 3: continue # prints: 0 1 2 4  
  
    print(i)
```

■■ CODE

```
students = ["kuldeep","rahul","sneha","sidu","anita"]  
  
for s in students: # break at sidu  
    if s == "sidu": break  
    print(s)  
  
for s in students: # skip sidu  
    if s == "sidu": continue  
    print(s)
```

18 TUPLES

■ DEFINITION

- Ordered, UNCHANGEABLE (immutable) collection using () brackets.
- Once created you CANNOT add, remove, or change items.
- Use when data should stay fixed. Indexing works like lists.

■ EXAMPLE

```
days = ("Mon", "Tue", "Wed")  
  
days[0] # "Mon"  
  
days[-1] # "Wed"  
  
days[0:2] # ("Mon", "Tue")
```

■■ CODE

```
my_tuple = (1, 2, 3, 4, 5)

print(my_tuple[0]) # 1

print(my_tuple[-1]) # 5

print(my_tuple[1:4]) # (2, 3, 4)
```

19 SET

■ DEFINITION

- Unordered collection of UNIQUE items using { } brackets.
- Duplicate values are removed automatically.
- Does NOT support indexing. Good for storing unique values.

■ EXAMPLE

```
s = {1, 2, 2, 3}

print(s) # {1, 2, 3} (duplicate removed)

s.add(4) # {1, 2, 3, 4}

s.remove(2) # {1, 3, 4}
```

■■ CODE

```
my_set = {1, 2, 3, 4, 5}

my_set.add(6)

my_set.remove(3)

print(my_set)

print(len(my_set))
```

20 DICTIONARY

■ DEFINITION

- Stores data as KEY-VALUE pairs using { } brackets.
- Each key is unique. Use key to access, update, or add values.
- dict["key"] → access value
- dict["key"] = val → update or add

■ EXAMPLE

```
student = {"name": "Rahul", "marks": 90}

student["name"] # "Rahul"

student["marks"] = 95 # update marks

student["grade"] = "A" # add new key
```

■■ CODE

```
my_dict = {"name": "kuldeep", "age": 20, "city": "bhopal"}

print(my_dict["name"]) # kuldeep

my_dict["age"] = 21 # update

my_dict["country"] = "india" # add new

print(my_dict)
```

21 FUNCTIONS

■ DEFINITION

- A reusable block of code that runs only when CALLED.
- def keyword → defines the function.
- parameters → inputs given inside () when calling.
- Avoids repeating the same code again and again.

■ EXAMPLE

```
def say_hello(name):

print("Hello", name)

say_hello("Rahul") # Output: Hello Rahul

say_hello("Sneha") # Output: Hello Sneha
```

■■ CODE

```
def greet(name):

print("Hello, " + name + "!")

greet("kuldeep")


def print_sum(a, b):

print("the sum is:", a + b)

print_sum(10, 5)

print_sum(20, 30)

print_sum(-5, 15)
```